

NLP



Feature Extraction and Preprocessing

A.D. Dulani S De Silva

What is NLP Preprocessing

It is the first step in any NLP task.

- Convert raw text into a clean and structured format.
- Remove unwanted elements like, symbols, spaced and inconsistent formats.
- Machines are unable to understand text directly
 - So we prepare it before giving it to models.

Example

“Hello!!! I LOVE NLP 🥰” → “hello i love nlp”

Why preprocessing is important?

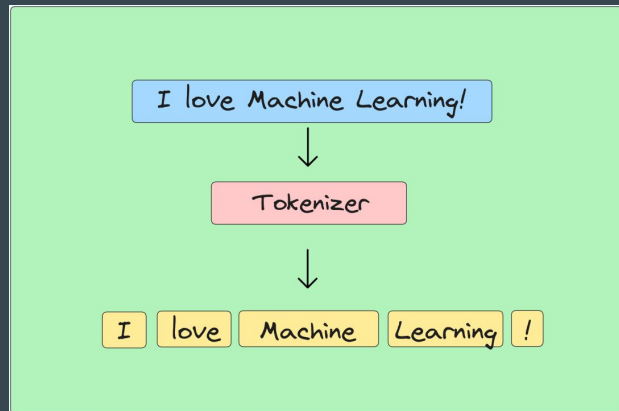
- Improves model performance by removing unnecessary data
 - Reduces noise in the dataset
 - Makes text consistent (same format everywhere)
 - Helps reduce complexity and computation time
-
- Key idea:
 - Cleaner data → Better model results

Types of Text Data Issues

- Noise: unwanted characters (symbols, HTML tags, punctuation)
- Ambiguity: same word has multiple meanings
 - Example:
 - “bank” (river bank / money bank)
- Sparsity: too many unique words, many appear rarely

Tokenization

- Splitting text into smaller units called tokens
- Tokens can be words, sentences or subwords
- Example
 - I love NLP => [I, love, NLP]
- Word Tokenization - splitting text into individual words
 - I am learning NLP => [I, am, learning, NLP]
- Sentence Tokenization - Split paragraph into sentences
 - I am learning NLP. It is interesting => [sentence1, sentence 2]
- Subword Tokenization - Break words into smaller meaningful parts, Helps to handle unknown or rare words
 - Unhappiness => [un, happy, ness]
 - Method : WordPiece, Byte Pair Encoding (BPE)



Tokenization

- Tokenizing English Challenges
 - “He didn’t want to pay \$50 for the book.”
 - Use python split() => separates text based on the spaces.
 - Output => ["He", "didn't", "want", "to", "pay", "\$50", "for", "the", "book."]
 - \$50 ??
 - book. ??
 - What if
 - “Let’s go to N.Y.C. for the weekend”
 - We need to handle special cases properly.
 - What is a word?
 - Is “full moon” one word or two words.
 - Should contractions such as don’t be split into two?

Tokenization

- Definition of word:
 - The smallest unit of speech that carries some meaning on its own.
- Related concepts
 - Morpheme: the smallest unit of speech which has a meaning, but does not necessarily stand on its own.
 - Example: -ing, re-, pre-, un-
 - Grapheme: the smallest functional unit of a writing system .
 - In english , that's letters.

Tokenization

- Implementation in Google Colab.

Text Cleaning

- Lowercasing (case normalization)
- Removing punctuation
- Removing numbers
- Removing special characters
- Handling whitespace

Lowercasing

- Convert all text into lowercase
- This avoids treating same words differently
- Example:
 - “Apple” and “apple” should be treated the same

Removing Punctuations

- Remove symbols like ., !, ?
- These usually do not add meaning in many tasks
-
- Example:
 - “Hello!!!” → “Hello”
-
- Note:
 - In sentiment tasks, punctuation can be useful

Handling Whitespace

- Remove extra spaces, tabs, and line breaks
- Keep only single spacing between words
-
- Example:
 - “Hello world” → “Hello world”

Stopword Removal

- Remove very common words that do not add much meaning
- Example
 - “I am going to school” → “going school”
- Words that appear very frequently (They usually do not carry strong meaning)
- Examples: the, is, in, at, and
- When to Remove / Not Remove
 - Remove - Text classification, Topic modeling
 - Not - Sentiment analysis, Tasks where meaning depends on them
 - “not good” → removing “not” changes meaning

Text Normalization

- Convert words into a standard form
- Reduces variations of the same word
- Methods
 - Stemming
 - Lemmatization
 - Handling contractions

Stemming

- Reduce words to root form by cutting endings
 - Removing word suffixes (and sometimes prefixes) {ing, s, y, ed}
- Fast but not always accurate
- Example
 - “running” → “run”
 - “studies” → “studi”
- Typically done through an algorithm, the most famous of which is Porter’s algorithm.

Lemmatization

- Convert words into correct base form using meaning
 - Reduce a word down to its lemma, or dictionary form
- More accurate than stemming
- Example:
 - “running” → “run”
 - “better” → “good”
 - “Did, Done, Doing” -> Do

Handling Contractions

- Expand short forms into full words
-
- Examples:
 - “don’t” → “do not”
 - “I’m” → “I am”
- Helps improve consistency

Handling Text Variations

- Real-world text is not clean
 - Spelling correction
 - Handling slang and abbreviations
 - Handling emojis and emoticons

Spelling correction

- Fix incorrect words
 - Example
 - “Recieve” => “receive”

Slang & Abbreviations

- Convert informal language into standard form
 - Example
 - U => you, brb => be right back

Emojis and Emoticons

- Convert emojis into text meaning
 - Example
 - 😊 → “happy” 😞 → “sad”

Feature Extraction Basics

- What is feature extraction
 - Feature extraction/engineering is process of converting raw text into numerical representations suitable for machine learning algorithms
 - AKA text representation or text vectorization.
- Importance: better features -> better performance
- Examples
 - Extracting word counts, POS tags, sentiment scores etc.

Role of feature engineering in NLP

- Utterly important in ML applications, where feature engineering is human centered.
- Picking unimportant features and omitting important features may cause inaccurate results.
- Challenge: preserving meaning while simplifying text.
- Features for text such as number of positive words, number of negative words, average word count etc.

Role of feature engineering in NLP

What we have: variable-length sequences of symbols.

What machine learning (ML) algorithms require: fixed-length numeric vectors.

['For', 'the', 'last', '8', 'years', 'of', 'his', 'life', ...]

['Edward', 'Teller', '&', 'this', 'man', 'partnered', ...]

['The', 'city', 'of', 'Yuma', 'in', 'this', 'state', 'has', 'a', ...]



[3, 45, 2, 6, 2, 5, 1, 45, 32, 9]

[6, 22, 4, 12, 7, 9, 3, 1, 19, 8]

[1, 7, 65, 4, 12, 9, 43, 1, 5, 6]

“Vectorization”

Core Idea

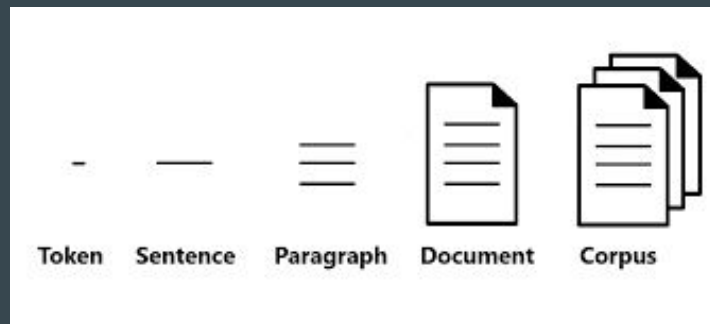
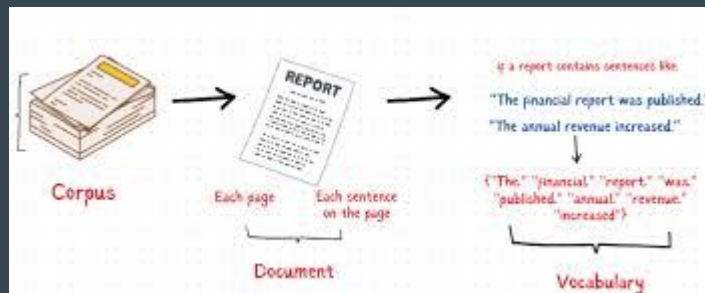
- Rule/ formula behind conversion of text to numbers
- Should capture semantic meaning otherwise it will not perform as successfully as needed.
- One hot encoding (OHC)
- Bag of Words (BoW)
- N-grams
- TF-IDF
- Custom features
- Word2vec and embeddings

Challenges in FE for NLP

- Out-of-vocabulary words
- Multilingual data
- Sarcasm, idioms, slangs
- Large model size and computational cost.

Common terms

- **Corpus (C)**
 - A large, structured collection of text or speech data used to train and develop language models. Generally, a large corpus provides more data, leading to better model performance
- **Vocabulary(V)**
 - Refers to the set of all unique tokens (typically words) found within a specific dataset or corpus.
- **Document (D)**
 - Document is a distinct, self containing unit of text, such as an article, email or even a sentence depending on the task (eg: each distinct review)
- **Word (W)**
 - Each word obtained through word embeddings are numerical vectors that capture semantic and syntactic relationships.

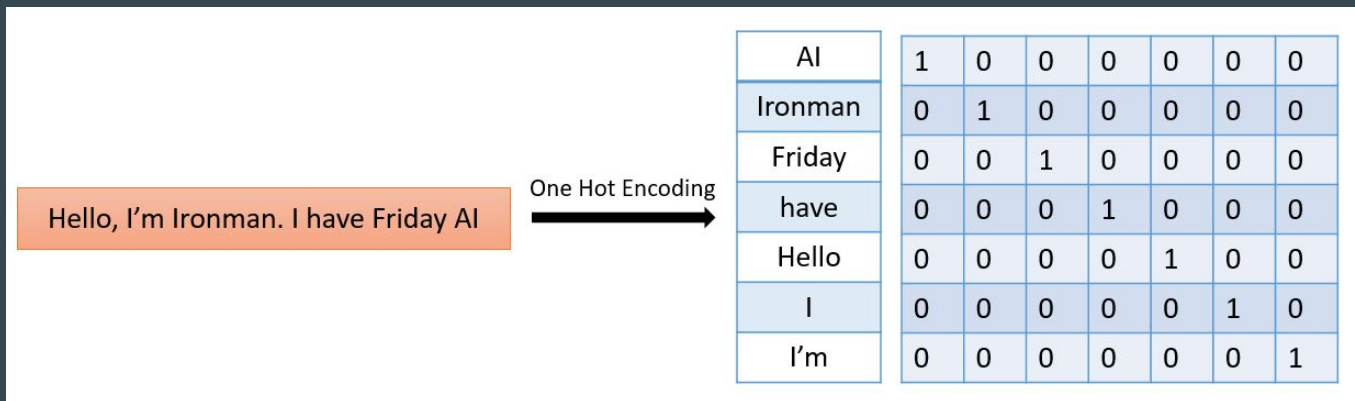


One Hot Encoding (OHC)

- Target is to convert these text to numbers
- Convert these words to vector representation
- In OHC, each word is converted to V dimensional vector.
- Pros
 - Intuitive
 - Easy to implement
- Cons
 - Sparsity (mostly zeros)
 - Impractical for large datasets because equal to vocabulary size
 - No fixed size
 - Out of vocabulary
 - No capturing of semantic meaning.

One Hot Encoding (OHC)

- Example



No Fixed Size

- OHC does have a fixed size but the problem is when it encounters another dataset with different size.
- Once $V = 10,000$ every one hot vector has the same fixed length i-e 10,000.
- Example
 - Dog = [1, 0, 0, ...]
 - Cat = [0, 1, 0, 0, 0, ...]
- Both vectors are 10,000 dimensional
- But when the model is trained on another dataset with different vocabulary size of vectors is changed.
 - Example
 - Dataset A: 10,000 unique words, vector length = 10,000
 - Dataset B: 50,000 unique words, vector length = 50,000
- So you can't easily transfer or reuse models trained on one dataset, the vector space changes completely.
- This is called “no globally fixed size” or “scalability issue”.

Out of vocabulary

- Suppose model was trained on this small vocabulary.
 - ["dog", "cat", "car", "tree"]
- New test sentence is: "The lion chased the cat"
- Here lion is not in the vocabulary, hence called OOV word.
- OOV are words not in model's own vocabulary.
- Due to language evolution, OOV words occur frequently.
- One hot models completely ignore OOV words.
- Performance of such models drop because OOV words may carry important meanings.

Bag of Words (BoW)

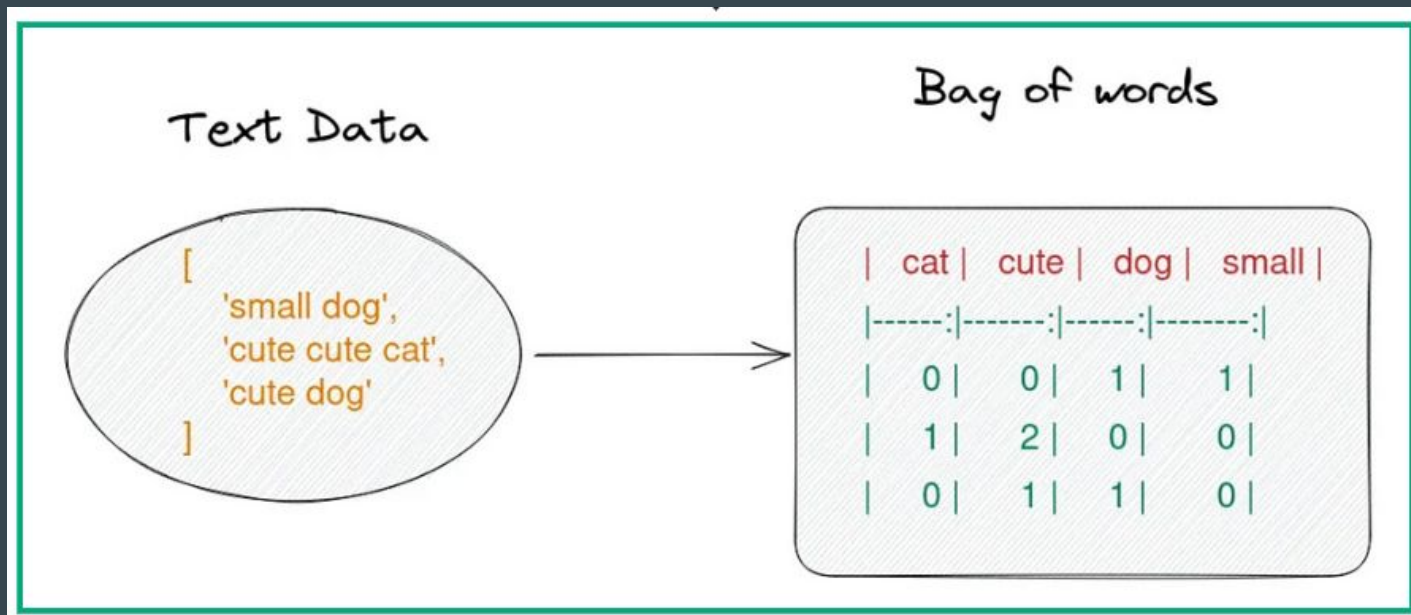
- Simplest technique in NLP.
- Creates a collection (bag) of words
- Each document/ text is represented as a word frequency vector where each dimension corresponds to a word in the vocabulary, and the value in that dimension is the count of how many times that word appears in the document.
- Creates a vocabulary of unique words from the text.
- Pros
 - Works well for small datasets
 - Easy to interpret
 - Useful for text classification
-

Bag of Words (BoW)

- Cons:
 - The model completely ignore word order, syntax and semantics
 - Ordering is also not considered
 - Creates a large sparse dataset
- Example: “I like NLP” -> [1, 1, 1]
-

Bag of Words (BoW)

- Example



BoW Model

- First a bag of words model is created, called Countvectorizer
- CountVectorizer creates a “bag of words” model where each unique word on the text becomes a feature and the value in the matrix for each document is the count of how many times that word appears in it.

BoW Model

Bag of Words Model explanation

Corpus
A collection of text documents



Tokenize
Divide the text into smaller units called tokens, usually words or phrases



Count word frequencies
Create a vocabulary of unique words and count the number of times each word appears in each document.



Encode the data
Encoding the text data as numerical values by creating a vector for each document, with each element of the vector representing the frequency count of a particular word in the document

Example

Corpus:
The dog is happy. The child makes the dog happy. The dog makes the child happy



Tokenization:
D1: [The] [dog] [is] [happy]
D2: [The] [child] [makes] [the] [dog] [happy]
D3: [The] [dog] [makes] [the] [child] [happy]



Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1



Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

© AIML.com Research

Binary BoW Example

d_1 : ['Red', 'Bull', 'drops', 'hint', 'on', 'F1', 'engine']

d_2 : ['Honda', 'exits', 'F1', 'leaving', 'F1', 'partner', 'Red', 'Bull']

d_3 : ['Hamilton', 'eyes', 'record', 'eighth', 'F1', 'title']

d_4 : ['Aston', 'Martin', 'announces', 'sponsor']

} Let's say we have
four documents...



['Aston', 'Bull', 'F1', 'Hamilton', 'Honda', 'Martin', 'Red',
'announces', 'drops', 'eighth', 'engine', 'exits', 'eyes', 'hint',
'leaving', 'on', 'partner', 'record', 'sponsor', 'title']

} Our resulting 20-
word vocabulary
(sorted)

Binary BoW Example

{

```
'Aston': 0,  
'Bull': 1,  
'F1': 2,  
'Hamilton': 3,  
'Honda': 4,  
'Martin': 5,  
'Red': 6,  
'announces': 7,  
'drops': 8,  
'eighth': 9,  
'engine': 10,  
'exits': 11,  
'eyes': 12,  
'hint': 13,  
'leaving': 14,  
'on': 15,  
'partner': 16,  
'record': 17,  
'sponsor': 18,  
'title': 19
```

d_1 : ['Red', 'Bull', 'drops', 'hint', 'on', 'F1', 'engine']

d_2 : ['Honda', 'exits', 'F1', 'leaving', 'F1', 'partner', 'Red', 'Bull']

d_3 : ['Hamilton', 'eyes', 'record', 'eighth', 'F1', 'title']

d_4 : ['Aston', 'Martin', 'announces', 'sponsor']

Resulting binary BOW encoding

D_1 : [0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0]

D_2 : [0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0]

D_3 : [0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1]

D_4 : [1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]

}

Frequency BoW Example

```
{  
'Aston': 0,  
'Bull': 1,  
'F1': 2,  
'Hamilton': 3,  
'Honda': 4,  
'Martin': 5,  
'Red': 6,  
'announces': 7,  
'drops': 8,  
'eighth': 9,  
'engine': 10,  
'exits': 11,  
'eyes': 12,  
'hint': 13,  
'leaving': 14,  
'on': 15,  
'partner': 16,  
'record': 17,  
'sponsor': 18,  
'title': 19
```

d_1 : ['Red', 'Bull', 'drops', 'hint', 'on', 'F1', 'engine']

d_2 : ['Honda', 'exits', 'F1', 'leaving', 'F1', 'partner', 'Red', 'Bull']

d_3 : ['Hamilton', 'eyes', 'record', 'eighth', 'F1', 'title']

d_4 : ['Aston', 'Martin', 'announces', 'sponsor']

Resulting binary BOW encoding

D_1 : [0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0]

D_2 : [0, 1, 2, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0]

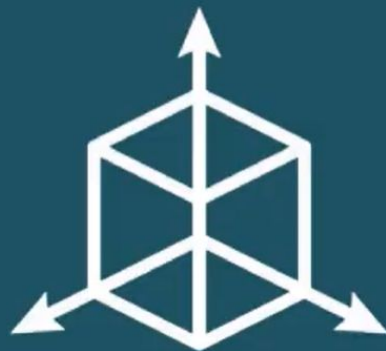
D_3 : [0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1]

D_4 : [1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]

We have shifted our thinking about text...



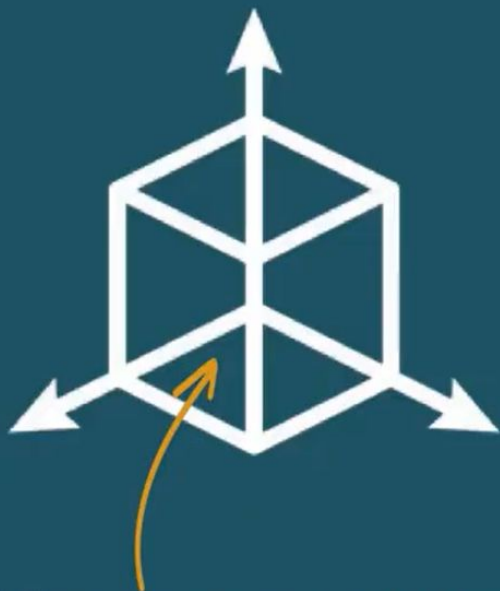
From a sequence of symbols...



to points in a multidimensional space that encodes some meaning of the text.

With BOW, we assume meaning is encoded in the presence of certain words.

We have shifted our thinking about text...

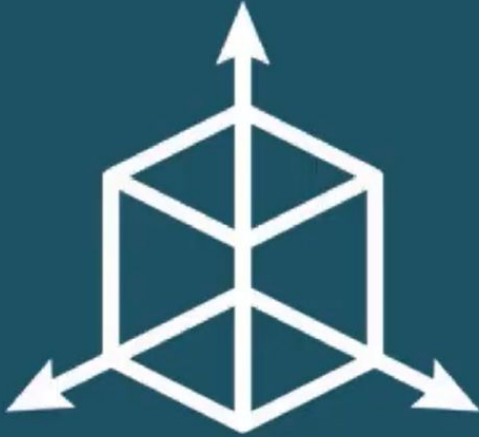


Each feature vector in our BOW is now a point in this multidimensional* space.

Vector Space Model (VSM)

*The number of dimensions is equal to the number of features (i.e. size of vocabulary).

Vector Space Model



What we can now do

Two documents which have similar vocabulary should be closer together in this space. This allows us to start measuring document similarity which is useful for:

- Relevance ranking
- Plagiarism detection
- Document classification

And other applications...

Dot Product

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n \underbrace{a_1 b_1 + a_2 b_2 + \dots + a_n b_n}$$

Two vectors of equal length (a, b)

Multiply their corresponding values and add the products together.

$$\begin{aligned} \mathbf{a} &= [1, 2, 3] \\ \mathbf{b} &= [4, 5, 6] \end{aligned} \quad \mathbf{a} \cdot \mathbf{b} = (1 \times 4) + (2 \times 5) + (3 \times 6) = 32$$

Dot Product

But high frequency words will lead to larger dot products...

So we normalize by
vector length:

$$\|\mathbf{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$$

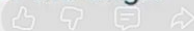
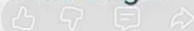


Which leads to

$$\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \cos \theta$$

Also known as the L^2 norm
or Euclidean norm.

The same as the cosine of the angle
between two vectors



Cosine Similarity

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

The value of **cos** θ ranges from 0 to 1.*

- 1 when the vectors point in the same direction.
- 0 when the vectors are orthogonal (dissimilar).

Cosine Similarity

$\mathbf{d}_1$: ['Red', 'Bull', 'drops', 'hint', 'on', 'F1', 'engine']

[0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0]

$\mathbf{d}_2$: ['Honda', 'exits', 'F1', 'leaving', 'F1', 'partner', 'Red', 'Bull']

[0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0]

$\mathbf{d}_3$: ['Hamilton', 'eyes', 'record', 'eighth', 'F1', 'title']

[0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1]

$\mathbf{d}_4$: ['Aston', 'Martin', 'announces', 'sponsor']

[1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]

$$\cos \theta = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\| \|\mathbf{d}_2\|} = 0.43$$

$$\cos \theta = \frac{\mathbf{d}_1 \cdot \mathbf{d}_3}{\|\mathbf{d}_1\| \|\mathbf{d}_3\|} = 0.15$$

$$\cos \theta = \frac{\mathbf{d}_1 \cdot \mathbf{d}_4}{\|\mathbf{d}_1\| \|\mathbf{d}_4\|} = 0.0$$

Drawbacks of BoW

- Does not capture similarity between synonyms
- No way to handle Out-of-vocabulary words
- Creates sparse vectors which can be inefficient.
- Word order information is lost.
 - We can overcome this using n-grams

N-grams (Bigrams, Trigrams, Unigrams etc)

- Why do we need n-grams?
 - Language is sequential - words depend on previous ones
 - We need a way to represent this context mathematically.
 - N-grams help capture local dependencies in text
- Example
 - “I love eating ice cream”
 - Bigram model sees
 - (“ I love”)
 - (“ love eating”)
 - (“ eating ice”)
 - (“ ice cream”)

N-grams

- An n-gram is a sequence of n consecutive items (usually words or characters) from a given text.
- Example
 - Unigram ($n=1$): “I”, “love”, “ice”, “cream”
 - Bigrams ($n=2$): “I love”, “love ice”, “ice cream”
 - Trigrams ($n=3$): “I love ice”, “love ice cream”
- BoW is a special case of bag of n-grams
- N-grams capture context by grouping words that appear together.

N-grams

- Pros
 - Able to capture semantic meaning of the sentence.
 - Easy to implement.
- Cons
 - Slows down the algorithm by increasing dimensions
 - No solution for OOV words.

Issues

- Binary BOW - All words are equally important.
- Frequency BOW - Words occur frequently and are not informative.

How about using relative frequency instead?

Issues

How about using relative frequency instead?



Given a word...



Five document icons are shown in a row. The first two have a blue checkmark, and the last three have a red X.

If a word occurs frequently in one or few documents but not in the rest, it's likely meaningful for those documents and should be considered more "important".

Given a word...



Five document icons are shown in a row, all with a blue checkmark.

But if that word occurs frequently across the corpus, then it should be considered **less** "important" and meaningful.

TF-IDF (Term Frequency - Inverse Document Frequency)

- TF-IDF builds upon the Bag of Words concept by introducing a weighting scheme that reflects a word's importance not only within a document but also across the entire corpus.
- TF-IDF enhances the BoW model by adding layer of consideration: the rarity of the word across the entire dataset. It gives unique words higher importance.
- It down wights common words (like “the”, “a”, “is”) that appear frequently in many documents, while giving more importance to words that are frequent in a specific document but rare across the corpus.

Term Frequency (TF)

$$\text{tf}(t, d) = f_{t,d}$$

Given a word(**t**) in a document(**d**)...

The *term frequency* is just how many times the term occurs in the document.

TF-IDF (Term Frequency - Inverse Document Frequency)

But some documents can result in much higher frequencies...

$$\text{tf}(\text{"dog"}, D_1) = 10$$

$$\text{tf}(\text{"dog"}, D_2) = 1000$$



Doesn't necessarily mean "dog" is that much more meaningful in D_2 . The document could just be longer.

TF-IDF (Term Frequency - Inverse Document Frequency)

So TF is often scaled to dampen the effect

$$\text{tf}(t, d) = \log_{10}(f_{t,d} + 1)$$

We take the log instead
(doesn't have to be base 10)

We also add a 1 in case $f_{t,d}$
turns out to be zero.

TF-IDF (Term Frequency - Inverse Document Frequency)

So TF is often scaled to dampen the effect

$$\text{tf}(t, d) = \log_{10}(f_{t,d} + 1)$$

$$\text{tf}(t, d) = \log_{10}(0 + 1) = 0$$

$$\text{tf}(t, d) = \log_{10}(100 + 1) = 2.0$$

$$\text{tf}(t, d) = \log_{10}(10000 + 1) = 4.0$$

TF-IDF (Term Frequency - Inverse Document Frequency)

Inverse Document Frequency (IDF)

$$\text{idf}(t, D) = \log_{10} \left(\frac{N}{n_t} \right)$$

N is the number of documents.

n_t is the number of documents t appears in.

Given a term(t) and a **corpus**(D)...

We take the log here as well.

TF-IDF (Term Frequency - Inverse Document Frequency)

Inverse Document Frequency (IDF)

$$\text{idf}(t, D) = \log_{10} \left(\frac{N}{n_t} \right)$$

N is the number of documents.

n_t is the number of documents t appears in.

Intuition

The fewer documents the term t appears in, the *higher* this IDF value will be, and the more weight it'll give to the word.

TF-IDF (Term Frequency - Inverse Document Frequency)

Final TF-IDF score for a term in a document

$$w_{t,d} = \underbrace{\text{tf}(t, d)} \times \underbrace{\text{idf}(t, D)}$$

The more frequently a term
appears in a given document...

...and the fewer times it
appears in other documents...



The higher its TF-IDF value.

TF-IDF (Term Frequency - Inverse Document Frequency)

d_1 : "you have brains in your head"

d_2 : "you have feet in your shoes"

d_3 : "you can steer yourself"

d_4 : "any direction you choose"

TF-IDF (Term Frequency - Inverse Document Frequency)

d_1 : "you have brains in your head"

d_2 : "you have feet in your shoes"

d_3 : "you can steer yourself"

d_4 : "any direction you choose"

d_1	TF	IDF	TF-IDF
"you"	0.3	0	0
"brains"	0.3	0.6	0.18

TF-IDF

- Term Frequency (TF): Measures how frequently a term appears in a document. Values within range (0, 1) hence aka probability.

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

- Inverse Document Frequency (IDF): Measures how rare a term is across the entire corpus. It calculated by taking the logarithm of the total number of documents divided by the number of documents containing the term.

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

- TF-IDF calculator: The TF-IDF score for a term in a document is the product of its TF and IDF values.

TF-IDF

- Implementation

TF-IDF

- Pros
 - Highlights informative words
 - Used in information retrieval systems, ie: google
- Cons
 - Still ignores context
 - Sparsity
 - Dimensions
 - Suffers OOV problem still.

Custom Features

- Features you can create yourself.
- No. of positive words, no. of negative words, ratio of positive to negative words, word count, character count etc.
- Usually NLP models are hybrid, having both features created using techniques as well as custom.

Word Embeddings

- In NLP word embeddings is a term used for the representation of words for text analysis, technically in the form of a real valued vector that encodes the meaning of the word such that the words that are closer in the vector space are expected to be similar in the meaning.
- Frequency based
 - Bag of Words
 - TF-IDF
 - GloVe
- Prediction based
 - Word2Vec

Word2Vec

- Deep learning based NLP technique.
- How it is different from previous.
- Able to capture semantic meaning (eg: “good” and “nice” are similar words)
- Able to create low dimensional vectors ([100-300]) than high dimensional vectors BoW and TF-IDF ([10,000])
- Saves computational power and time
- It creates dense (non-zero) vectors than sparse vectors
- Dense vectors helps avoid overfitting problem.

Word2Vec

- Underlying assumption is that two words sharing similar context also share a similar meaning and consequently a similar representation in vector space.
- Words with similar meanings have close vectors.
- Example
 - King - man + women = queen
- Pros
 - Capture semantic relationships
- Cons
 - Fixed meaning of each word (context sensitive)

Problem

- Practical dataset is very huge.
- Handcraft such features for huge dataset is very difficult
- Automate creation of features, hence NN is needed.
- Downside is that you cannot justify values in vector because they are created by the ANN.

Types of Word2Vec

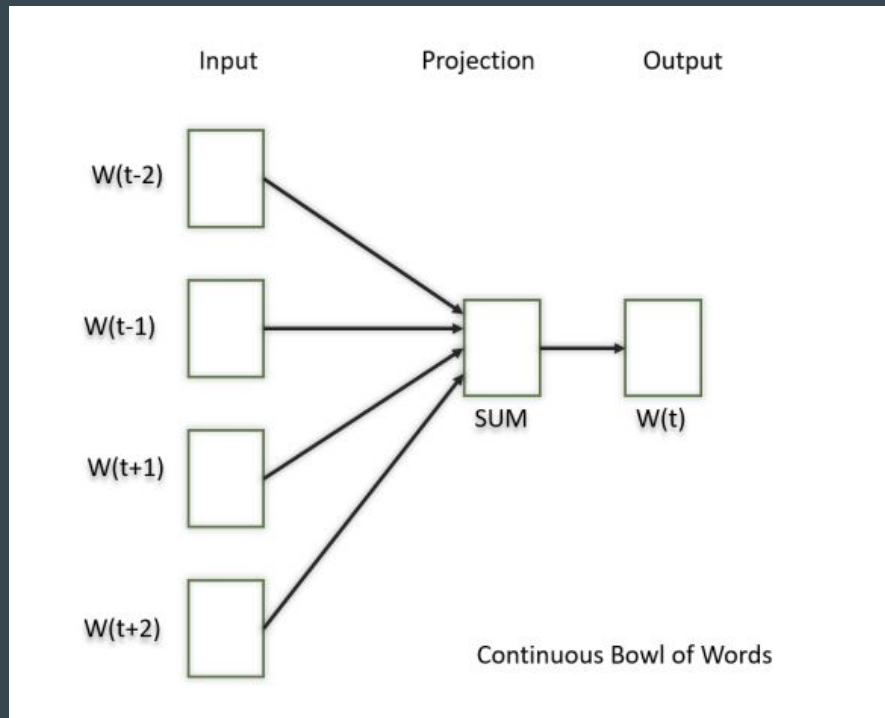
- Both are shallow NN (does not have many layers)
- CBoW (Continuous Bag of Words)
- Skip-gram

CBoW

- A type of neural network architecture used in the Word2Vec model.
- The CBOW model learns to predict a target word given its surrounding context words in a given window.
- Given a sequence of words in a context window, the model is trained to predict the target word at the center of the window.
- Fake problem -> Solve -> Vector
- Byproduct of fake/dummy problem

CBoW

- Feedforward neural network with a single hidden layer.
- The input layer, hidden layer, and output layer represent the context words, learned continuous vectors or embeddings, and the target word, respectively.
- Useful for learning distributed representations of words in a continuous vector space.



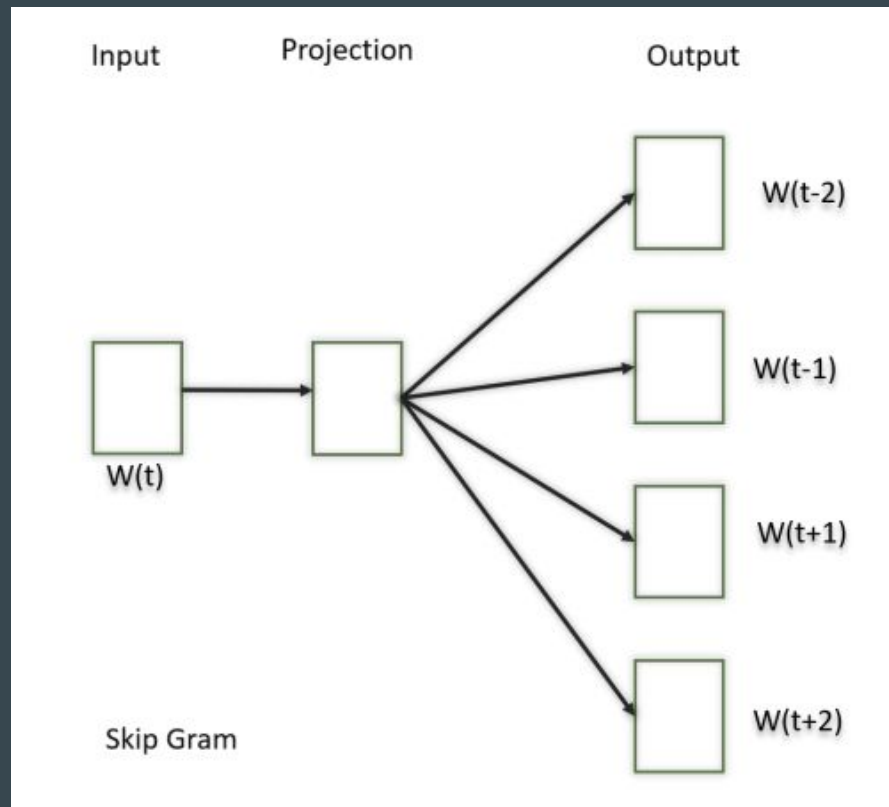
- The hidden layer contains the continuous vector representations (word embeddings) of the input words.
- The weights between the input layer and the hidden layer are learned during training.
- The dimensionality of the hidden layer represents the size of the word embeddings (the continuous vector space).

Skip-gram

- This model learns distributed representations of words in a continuous vector space.
- The main objective of Skip-Gram is to predict context words (words surrounding a target word) given a target word. This is the opposite of the Continuous Bag of Words (CBOW) model, where the objective is to predict the target word based on its context.
- It is shown that this method produces more meaningful embeddings.

Skip-gram

- Output: Trained vectors of each word after many iterations through the corpus.
- Preserve syntactic or semantic information, Converted to lower dimensions.
- Similar meaning (semantic info) vectors are placed close to each other in space.
- `vector_size` parameter controls the dimensionality of the word vectors, and you can adjust other parameters such as `window`.



Skip-gram

- The choice between CBOW and Skip-gram depends on data and the task.
- CBOW might be preferred when training resources are limited, and capturing syntactic information is important.
- Skip-gram is chosen when semantic relationships and the representation of rare words are important.

Contextual Embeddings (BERT, GPT)

- Context matters: “bank” (river vs money)
- Models like bert, gpt produce dynamic embeddings
- Each word’s meaning changes with sentence context.
- Example:
 - “I went to the bank” vs “He sat on the river bank”.

Feature Scaling and Selection

- Feature normalization
- Dimensionality reduction
- (PCA, Chi-Square, Feature Selection)

End of the Lecture